

REST and Hypermedia

Prof. Dr. Peter Braun

6. Januar 2025

1 Titlepage

REST and Hypermedia

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Learning Goals

Learning Goals

- You learn about the hypermedia principle of REST
- You learn about the 5-URLs-Pattern
- You learn how to define hyperlinks in responses

In this unit, we will explore another crucial, if not the most crucial, property of REST: the hypermedia property. To grasp this concept, we must first recall the structure of the World Wide Web. The World Wide Web consists of documents distributed across numerous web servers and made available on demand. Individual documents can be interconnected through hyperlinks. When a user views a document in a web browser, they see links to additional documents, often in the form of highlighted words, for example. If the user seeks more information about a topic, they can select the hyperlink by clicking on the word. The browser then generates a new request for the linked document, which may reside on another server, and subsequently displays this document. The server delivers the document in a specific representation and presents various options via hyperlinks regarding potential next actions with this document. The choice to select a hyperlink or which available hyperlinks to choose ultimately lies with the end-user. The server offers suggestions for the next steps, but it is the end-user who ultimately decides. We now aim to apply this principle of hyperlinks to the programming of our APIs.

3 Current State of the Person Service

Current State of the Person Service

- GET /demo/api/persons to fetch all persons
- GET /demo/api/persons/{id} to fetch one person
- POST /demo/api/persons to create a new person
- PUT /demo/api/persons/{id} to update one person
- DELETE /demo/api/persons/{id} to delete one person

We call this the **5 URLs pattern** of resources

First, let's revisit the current state of our development. We aim to implement a service that clients can use to manage people. We have identified a resource called "Person." Each person has standard attributes such as first name, last name, email address, and date of birth. Now that we understand the significance of the four HTTP verbs, our API provides these verbs as an interface. With the HTTP verb GET, the client can query all persons or retrieve a specific person if it knows the unique ID. The HTTP verb POST can create a new resource of the type Person. The HTTP verb PUT allows the client to update a resource, while the DELETE verb enables it to remove a resource. Rather than focusing on the four operations related to CRUD, we will now discuss the five standard URLs associated with the resource Person. Note that we have mentioned the HTTP operation GET twice here. This scenario is typical for REST applications, so let's temporarily refer to these five URLs as the "5 URL Pattern for Resources." Now, let's strive to clarify our understanding and identify any open questions by asking a few inquiries.

4 Question 1

Question 1

- From the perspective of clients and concerning the current state of the implementation: what URLs must be known to clients to let them work correctly?

First question: If you are now a client developer, which of these five URLs would you need to know to perform all the desired operations with a resource? Where in the source code would you need to store the full URL with hostname, port number, and full path to perform the operations?

5 Answer 1

Answer 1

- The client developer needs all five URLs

The correct answer is that you would need to know all five URLs. Some may think that the client only needs to understand the basic structure of the URL, including the path to reach a resource person within a URI. With this knowledge, the client developer could then determine all the other URLs independently. In principle, this is correct, but the assumption that these five URLs always follow the same scheme is not necessarily valid. The server developer may have used the singular URL path for reading, updating, and deleting a single person, i.e., Person, while still using the plural, Persons, for accessing all individuals. This deviation would disrupt your pattern. Typically, separate classes or methods exist in the client for the five accesses, and the URLs are fixed there, often as constant text.

6 Question 2

Question 2

- What happens if the name of resource `Person` is changed to `User` and the URLs are changed to `/demo/api/users`?

Second question: let's assume that the server developer would like to rename the resources; instead of "Person," the new name should be `User`. Consequently, the URLs would also change, and instead of the path element "Persons," he would now use the path element `Users`. What effects would this have on the client's programming?

7 Answer 2

Answer 2

- The client is broken and won't work anymore
- Modification of the source code necessary

The correct answer is that the client needs reprogramming. All hard-coded URLs in the client must be updated. If you were an experienced client developer, you would use methods that allow you to change the URL in one location in the code, making the effort to change it relatively minor. However, the client still needs to be recompiled, rebuilt, and made available again; otherwise, it will cease to function.

8 Question 3

Question 3

- From the perspective of clients: what is the URL that a client has to know when the application starts?

Question 3: Considering the drawbacks of a client programmer needing to know all five URLs, along with the concept of hyperlinks, please reassess: which URLs do the client programmer truly need to know?

9 Answer 3

Answer 3

- One URL should be enough – this must not change!
- It's like the home page of the API
- We call this the **dispatcher address**
- From there, the client gets other URLs as hyperlinks

The correct answer is that the client developer should rely on a single URL, which serves as the starting point for their application on the server. This entry address must remain stable; in other words, it should never change. When the client makes the initial request to this entry address, the server responds with a message containing hyperlinks that the client can use next. We refer to this entry point as the dispatcher. Let's compare this to how a human reader visits a website. If you want to access a news portal, you type the portal's entry address into the address bar of your web browser, such as www.spiegel.de. This is often referred to as the "home page" of the portal. From there, as a human user, you navigate through the hyperlinks to other pages, like individual articles. I would argue that, as a human user, you would never enter the full address of a specific article into your browser's address bar. This is precisely how we should approach programming APIs.

10 Dispatcher State

Dispatcher State

```
1 @Path( "" )
2 public class DispatcherService {
3     @Context
4     protected UriInfo uriInfo;
5
6     @GET
7     public Response getDispatcher() {
8         final Response.ResponseBuilder builder = Response.ok( );
9
10        Hyperlinks.addLink( uriInfo, builder, "/demo/api/persons/", "getAllPersons", "application/json" );
11
12        return builder.build( );
13    }
14 }
```

- The DispatcherService can be reached at /demo/api
- **Clients only need to know this single URL!**

How can we implement a dispatcher? Let's examine the context path of the Demo, which has the path element API already configured for our REST application. Now, you can easily establish a dispatcher as a distinct service that contains only one method to handle the HTTP GET request. The server sends potential hyperlinks to the client in the response message, which the client can utilize. We don't include these hyperlinks in the content area of the HTTP response message; that area remains completely empty while we instead place the hyperlinks in the header. In our sample project, there's a class called Hyperlinks that simplifies the process for us as developers to add these hyperlinks to the header. Before we investigate the three elements of a hyperlink more closely, I want to emphasize that this URI to the dispatcher is the only address the client needs to interact successfully with our application.

11 Links in the Response Header

Links in the Response Header

- Links in the response header must follow the specification in RFC 5988

```
1 <http://localhost:8080/demo/api/persons>;rel="getAllPersons",type="application/json"
```

- `rel` = relation type, i.e. the meaning of the hyperlink
- `type` = media type, i.e. the data type to be expected or to be used
- Please note that we are using **absolute** URLs here (not relative)!

There is a standard for representing hyperlinks in the headers of HTTP response messages that we will follow here. According to this standard, a hyperlink consists of three elements. The first element is the URI, which must be absolute; it should include the full address with the hostname and port number rather than just a relative one with path information. The second element describes the relation type, communicating the meaning of this link. This is simply a text field that allows for a free choice of the relation type; currently, there are no standards or specifications that dictate how the relation type should be structured. For simplicity, we use a human-readable description to clarify the use case and meaning of this hyperlink. From the brief description “getAllPersons,” it should be clear that this hyperlink is intended for clients who wish to query the list of all persons. The final element specifies the media type that the client must use when sending the next request. Thus, the server informs the client of the media type to use next.

12 What Hyperlinks Should be Provided by the Server?

What Hyperlinks Should be Provided by the Server?

- For every request that clients send to the server, the server should provide hyperlinks

Now you understand how the server uses hyperlinks to inform the client about the next possible steps. You also have a preliminary understanding of how the server technically transmits these hyperlinks to the client. In the next unit, we will discuss in detail what hyperlinks the server should respond with.

13 Hyperlinks in the Response Body

Hyperlinks in the Response Body

```
1 public class Person {  
2     private long id;  
3     private String firstName;  
4     private String lastName;  
5  
6     private Link self;  
7  
8     public Link getSelfLink() {  
9         return self;  
10    }
```

- The self-link must be set in the class that handles the HTTP verbs and creates the responses

At this point, I would like to briefly explain how you can incorporate such links into a resource's representation. Unfortunately, it's not that simple. The source code beside me demonstrates how to define an attribute for the self-link. The class "Link," used here in line 6, is defined in the Quarkus Demo project. A class is responsible for handling all incoming HTTP requests and producing responses. In the Quarkus Demo project, this class is called "UniversityWebController." Whenever a method returns a resource representation, the self-link attribute is manually set to the absolute path of this resource, including the ID. If you opt for another web framework, such as Spring, things might be even more challenging.

14 Consequences

Consequences

- Clients only know one single URL of the server
- Server responds hyperlinks to possible next actions
- Clients pick one of the provided hyperlinks

To summarize, clients require only one URL, which is the one from the dispatcher. For every request the client sends to the server, the server responds with hyperlinks that guide the client on what to do next. These hyperlinks are usually stored in the header of the response message. In a Get Collection access, links to individual resources are included in the content area as `self` links. The server provides these links to determine whether a client can perform specific actions. If the server does not provide a hyperlink for updating a resource, the client cannot execute that update. Consider the client programming, whether it is a mobile application or a web app. A button or icon could represent a possible action for updating a resource. For the client developer, deciding whether to permit the end-user to update the resource is straightforward. They only need to check if they have received the appropriate link from the server. If the link is included in the response message, the button is displayed; if not, it is hidden.

15 Summary

Summary

- The most important principle of REST is using hyperlinks to let the client be driven through the application by the server
- **Hypermedia As The Engine Of Application State (HATEOAS)**
- In fact, it is just the basic principle of the World Wide Web
- Using HATEOAS is a **must** for a REST application
- If you don't provide hyperlinks, don't claim your system to be REST-ful

This brings us to the conclusion of this unit. Today, you have learned a very important—or perhaps even the most critical—principle in designing REST applications. In such an application, the client needs to know only one URI, which represents the application's entry point. It receives all other URIs as a result of requests. With each request, the server sends the client, usually in the header of the response message and, in one case, also in the content area, the URIs that can be used meaningfully next. Thus, we have established a fitting analogy to the basic principle of the World Wide Web. In both cases, the server provides data and offers the client options through hyperlinks about which actions can be carried out next. Of course, it is up to the client, and in most cases, the end user, to choose the specific next action. We could also express it this way: the client exists in a certain state at every moment of the application. At the beginning of the application, after launching it, the client is in the start state. The only useful operation it can perform at this moment is to call the dispatcher. As a result of this first request, the server provides the client with hyperlinks leading to the next state of the application. For example, the first screen in the client's application should now display the list of all individuals. Therefore, the client subsequently requests the collection of all resources. The client then reaches the next state when it shows an individual resource in detail, such as a new screen. From there, the user can choose whether to modify or delete a resource. However, this is only possible if the server has provided the corresponding hyperlinks. In a REST-based system, the server determines what the possible subsequent states are. The final decision regarding which potential subsequent state is ultimately chosen lies with the client or end user. Possible subsequent states are described through hyperlinks. By now, you may recognize the abbreviation HATEOAS: Hypermedia As The Engine Of Application State. Through hypermedia, specifically hyperlinks, the server guides the

client through the various states of the application. There is considerable debate about whether implementing this principle is essential to classify a system as RESTful. However, there is no doubt about this, as the term's inventor, Roy Fielding, has clearly indicated in numerous articles and blog posts that a REST system is only fully realized if the principle of HATEOAS has been correctly applied. It should also be noted that he dismisses the term.

16 Literature

Literature

- Bill Burke: RESTful Java with JAX-RS 2.0: Designing and Developing Distributed Web Services (Englisch). O'Reilly, 2013.